Experiment # 04

Implementation of Connected Component Analysis

Objective

1. To implement connected component labelling.
2. To count number of objects in an image.

Software

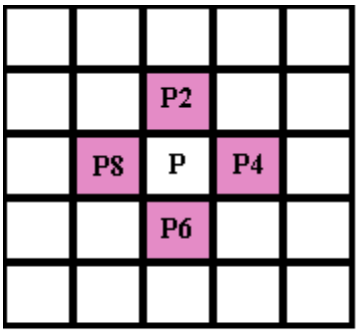
- Python Idle 3.7 or 3.8.5
- Spyder

Theory

A connected component is a set of black pixels, P, such that for every pair of pixels p_i and p_j in P, there exists a sequence of pixels p_i, ..., p_j such that:
a) All pixels in the sequence are in the set P i.e. are black, and
b) Every 2 pixels that are adjacent in the sequence are "neighbors"
This gives rise to 2 types of connectedness, namely: 4-connectivity and 8-connectivity.

4-Connectivity

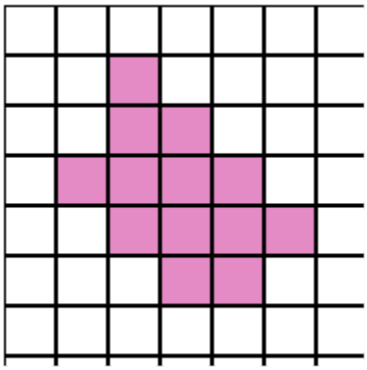
A pixel, Q, is a 4-neighbor of a given pixel, P, if Q and P share an edge. The 4-neighbors of pixel P (namely pixels P2, P4, P6 and P8) are shown in figure below.

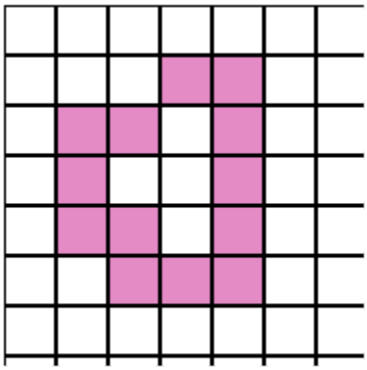


4-connected component:
A set of black pixels, P, is a 4-connected component if for every pair of pixels p_i and p_j in P, there exists a sequence of pixels p_i, ..., p_j such that:
a) All pixels in the sequence are in the set P i.e. are black, and
b) Every 2 pixels that are adjacent in the sequence are 4-neighbors

Examples of 4-connected patterns:

The following diagrams are examples of patterns that are 4-connected:

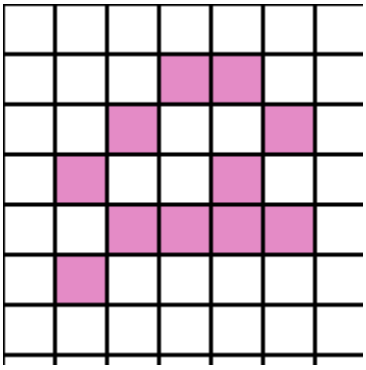




8-Connectivity

A pixel, Q, is an 8-neighbor (or simply a neighbor) of a given pixel, P, if Q and P either share an edge or a vertex.
A set of black pixels, P, is an 8-connected component (or simply a connected component) if for every pair of pixels p_i and p_j in P, there exists a sequence of pixels p_i, ..., p_j such that:
a) All pixels in the sequence are in the set P i.e. are black, and
b) Every 2 pixels that are adjacent in the sequence are 8-neighbors

Example of 8-connected pattern:
The diagram below is an example of a pattern that is 8-connected but not 4-connected:



Connected Component Analysis or Labelling enables us to detect different objects from a binary image. Once different objects have been detected, we can perform a number of operations on them: from counting the number of total objects to counting the number of objects that are similar, from finding out the biggest object of the bunch to finding out the smallest and from finding out the closest pair of objects to finding out the farthest etc.

Connected Component labelling procedure is as follows:

- Process the image from left to right, top to bottom:

  If the next pixel to process is 1

  i.) If only one from top or left is 1, copy its label.

  ii.) If both top and left are one and have the same label, copy it.

  iii.) If top and left they have different labels
  - Copy the smaller label
  - Update the equivalence table.

  iv.) Otherwise, assign a new label.

- Re-label with the smallest of equivalent labels

Video explanation: https://www.youtube.com/watch?v=ticZclUYy88

Some Useful Commands:

- To convert the image from RGB to gray sclae

  cv2.cvtcolor(image, cv2.color_bgr2gray)

- To convert image into binary color

  cv2.threshold(image, thresh, maxval, dst)

  ret, binary = cv2.threshold(s,127,255,cv2.THRESH_BINARY)

- To find form of some object

  cv2.findContours(image, mode, method)

  contours, hierarchy = cv2.findContours(binary,cv2.RETR_TREE,cv2.CHAIN_APPROX_SIMPLE)

- To draw contours

  cv2.drawContours(image, contours, contourIdx, color, thickness)

  image=cv2.drawContours(image,contours,-1,(0,0,255),3)

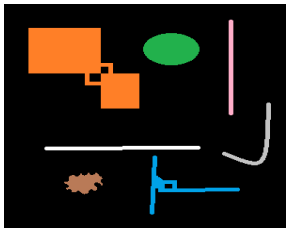
- To find connected components

  cv2.connectedComponents(image, connectivity)

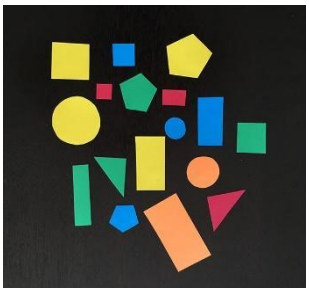
  r,c=cv2.connectedComponents(image, connectivity)

Lab Task:

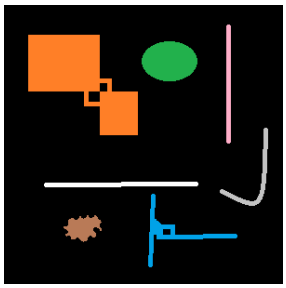
1. For the image given below (provided with the lab handout), apply the connectected component labelling and count the total number of colored objects.



2. First convert image given below in gray scale and threshold the images and then do connected component analysis. (Do this using build-in commands)



(a)



(b)